



Решения всех практических задач по циклам



Уровень 1: Начальный

Задача 1.1: Таблица умножения (for)

```
#include <iostream>
using namespace std;

int main() {
    int number = 7; // Можно запросить у пользователя

    cout << "Таблица умножения на " << number << ":" << endl;

    // Цикл от 1 до 10
    for(int i = 1; i <= 10; i++) {
        // Вычисляем произведение
        int result = number * i;
        // Выводим строку таблицы
        cout << number << " x " << i << " = " << result << endl;
    }

    return 0;
}
```

Улучшенная версия с вводом:

```
#include <iostream>
using namespace std;

int main() {
    int number;
```

```

cout << "Введите число для таблицы умножения: ";
cin >> number;

cout << "\nТаблица умножения на " << number << ":" << endl;
cout << "-----" << endl;

for(int i = 1; i <= 10; i++) {
    // Используем setw для красивого форматирования
    cout << number << " x " << i << " = " << number * i << endl;
}

return 0;
}

```

Задача 1.2: Сумма чисел (while)

```

#include <iostream>
using namespace std;

int main() {
    int number;    // Текущее введенное число
    int sum = 0;   // Сумма всех чисел (начинаем с 0)

    cout << "Вводите числа (0 для завершения):" << endl;

    // Бесконечный цикл, выходим по break
    while(true) {
        cout << "Введите число: ";
        cin >> number;

        // Проверяем, не пора ли закончить
        if(number == 0) {
            break; // Выходим из цикла
        }

        // Добавляем число к сумме
        sum += number; // То же что sum = sum + number
    }

    cout << "Сумма всех введенных чисел: " << sum << endl;
}

```

```
    return 0;
}
```

Альтернатива с проверкой в условии:

```
#include <iostream>
using namespace std;

int main() {
    int number, sum = 0;

    cout << "Введите первое число: ";
    cin >> number;

    // Пока число не равно 0
    while(number != 0) {
        sum += number; // Добавляем к сумме

        cout << "Введите следующее число (0 для завершения): ";
        cin >> number; // Читаем следующее число
    }

    cout << "Итоговая сумма: " << sum << endl;

    return 0;
}
```



Задача 1.3: Обратный отсчет (do-while)

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Введите число N для обратного отсчета: ";
    cin >> n;

    cout << "Обратный отсчет от " << n << " до 1:" << endl;
```

```
int counter = n; // Создаем копию для отсчета

do {
    cout << counter << endl; // Выводим текущее значение
    counter--;               // Уменьшаем на 1
} while(counter >= 1);      // Пока не дойдем до 1

cout << "Поехали!" << endl;

return 0;
}
```

Вариант с проверкой отрицательных чисел:

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Введите число N: ";
    cin >> n;

    if(n <= 0) {
        cout << "Число должно быть положительным!" << endl;
        return 1;
    }

    cout << "Обратный отсчет:" << endl;

    int i = n; // Начинаем с N
    do {
        cout << i << endl;
        i--;    // i = i - 1
    } while(i > 0); // Пока i больше 0

    cout << "Старт!" << endl;

    return 0;
}
```

Уровень 2: Средний

Задача 2.1: Поиск простых чисел

```
#include <iostream>
#include <cmath> // для sqrt
using namespace std;

int main() {
    int n;

    cout << "Введите N (верхнюю границу): ";
    cin >> n;

    if(n < 2) {
        cout << "Простые числа начинаются с 2!" << endl;
        return 1;
    }

    cout << "Простые числа от 2 до " << n << ":" << endl;

    // Перебираем все числа от 2 до N
    for(int i = 2; i <= n; i++) {
        bool isPrime = true; // Предполагаем, что число простое

        // Проверяем делители от 2 до корня из i (оптимизация)
        for(int j = 2; j * j <= i; j++) {
            // Если i делится на j без остатка
            if(i % j == 0) {
                isPrime = false; // Число не простое
                break;           // Выходим из внутреннего цикла
            }
        }

        // Если число простое - выводим его
        if(isPrime) {
            cout << i << " ";
        }
    }

    cout << endl;
```

```
    return 0;
}
```

Оптимизированная версия:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cout << "Введите N: ";
    cin >> n;

    // Решето Эратосфена - более эффективный алгоритм
    vector<bool> isPrime(n + 1, true); // Массив флагов

    // 0 и 1 - не простые
    isPrime[0] = isPrime[1] = false;

    for(int i = 2; i * i <= n; i++) {
        if(isPrime[i]) {
            // Вычеркиваем кратные i
            for(int j = i * i; j <= n; j += i) {
                isPrime[j] = false;
            }
        }
    }

    cout << "Простые числа: ";
    for(int i = 2; i <= n; i++) {
        if(isPrime[i]) {
            cout << i << " ";
        }
    }
    cout << endl;

    return 0;
}
```

Задача 2.2: Факториал с проверкой

```
#include <iostream>
using namespace std;

int main() {
    int n;
    long long factorial = 1; // long long для больших чисел

    cout << "Введите число N: ";
    cin >> n;

    // Проверка на отрицательное число
    if(n < 0) {
        cout << "Ошибка: факториал определен только для неотрицательных чисел." << endl;
        return 1;
    }

    // Проверка на большие числа
    if(n > 20) { // 21! уже не влезает в long long
        cout << "Внимание: результат может быть неверным для больших чисел." << endl;
    } else if(n > 10) {
        cout << "Внимание: результат будет большим числом." << endl;
    }

    // Вычисляем факториал
    // Способ 1: цикл for
    for(int i = 1; i <= n; i++) {
        factorial *= i; // factorial = factorial * i
        // Для отладки можно выводить промежуточные значения:
        // cout << i << "! = " << factorial << endl;
    }

    /* Способ 2: цикл while
    int i = 1;
    while(i <= n) {
        factorial *= i;
        i++;
    }
    */

    cout << n << "! = " << factorial << endl;
```

```
    return 0;
}
```

С вычислением промежуточных значений:

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Вычисление факториала" << endl;
    cout << "Введите число N: ";
    cin >> n;

    if(n < 0) {
        cout << "Факториал отрицательного числа не определен!"
        return 1;
    }

    cout << "\nПроцесс вычисления:" << endl;
    long long result = 1;

    for(int i = 1; i <= n; i++) {
        result *= i;
        cout << "Шаг " << i << ": " << i << "! = " << result;

        if(result < 0) {
            cout << " (переполнение!)";
        }
        cout << endl;
    }

    cout << "\nИтог: " << n << "! = " << result << endl;

    return 0;
}
```



Задача 2.3: break и continue

```
#include <iostream>
using namespace std;

int main() {
    int number;        // Вводимое число
    int sum = 0;        // Сумма подходящих чисел

    cout << "Вводите числа (отрицательное для выхода):" << endl;

    while(true) { // Бесконечный цикл
        cout << "Введите число: ";
        cin >> number;

        // Проверка на выход
        if(number < 0) {
            cout << "Обнаружено отрицательное число. Выход." << endl;
            break; // Немедленно выходим из цикла
        }

        // Проверка на кратность 3
        if(number % 3 == 0) {
            cout << "Число " << number << " кратно 3, пропускаем." << endl;
            continue; // Переходим к следующей итерации
        }

        // Если дошли сюда - число положительное и не кратно 3
        sum += number;
        cout << "Добавлено " << number << ". Текущая сумма: " << sum << endl;
    }

    cout << "\nИтоговая сумма: " << sum << endl;

    return 0;
}
```

Более компактная версия:

```
#include <iostream>
using namespace std;
```

```
int main() {
    int num, total = 0;

    cout << "Ввод чисел (для выхода введите отрицательное):" << endl;

    while(cin >> num && num >= 0) {
        if(num % 3 == 0) continue; // Пропускаем числа кратные 3
        total += num;
    }

    cout << "Сумма чисел (без кратных 3): " << total << endl;

    return 0;
}
```

Уровень 3: Продвинутый

Задача 3.1: Числа Фибоначчи

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Сколько чисел Фибоначчи вывести? ";
    cin >> n;

    if(n <= 0) {
        cout << "Количество должно быть положительным!" << endl;
        return 1;
    }

    long long a = 0, b = 1; // Первые два числа
    int count = 0;          // Счетчик выведенных чисел

    cout << "Первые " << n << " чисел Фибоначчи:" << endl;
```

```
// Способ 1: через while
while(count < n) {
    cout << a;
    if(count < n - 1) cout << ", ";

    // Вычисляем следующее число
    long long next = a + b;
    a = b;      // Сдвигаем значения
    b = next;   // для следующей итерации
    count++;    // Увеличиваем счетчик
}

cout << endl;

return 0;
}
```

Версия с for:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Введите N: ";
    cin >> n;

    if(n < 1) {
        cout << "N должно быть >= 1" << endl;
        return 1;
    }

    cout << "Числа Фибоначчи:" << endl;

    if(n >= 1) cout << "F(1) = 0" << endl;
    if(n >= 2) cout << "F(2) = 1" << endl;

    long long prev1 = 0, prev2 = 1;

    for(int i = 3; i <= n; i++) {
        long long current = prev1 + prev2;
        cout << "F(" << i << ") = " << current << endl;
    }
}
```

```
        // Обновляем значения для следующей итерации
        prev1 = prev2;
        prev2 = current;
    }

    return 0;
}
```

Задача 3.2: График функции

```
#include <iostream>
#include <cmath> // для pow
using namespace std;

int main() {
    cout << "График функции  $y = x^2$ " << endl;
    cout << "x от -5 до 5" << endl;
    cout << "=====" << endl;

    for(int x = -5; x <= 5; x++) {
        // Вычисляем  $y = x^2$ 
        int y = x * x;

        // Выводим координаты
        cout << "x=" << x << " y=" << y << " ";

        // Рисуем график звездочками
        // Ограничиваем количество звездочек для читаемости
        int stars = y / 2; // Масштабируем для экрана

        // Ограничиваем максимальную длину
        if(stars > 30) stars = 30;

        for(int i = 0; i < stars; i++) {
            cout << "*";
        }

        cout << endl;
    }
}
```

```
    return 0;  
}
```

Улучшенная версия с масштабированием:

```
#include <iostream>  
#include <iomanip> // для setw  
using namespace std;  
  
int main() {  
    const int MIN_X = -5;  
    const int MAX_X = 5;  
    const int MAX_STARS = 50;  
  
    cout << "График функции  $y = x^2$ " << endl;  
    cout << string(60, '=') << endl;  
  
    // Находим максимальное значение y для масштабирования  
    int maxY = 0;  
    for(int x = MIN_X; x <= MAX_X; x++) {  
        int y = x * x;  
        if(y > maxY) maxY = y;  
    }  
  
    cout << "Масштаб: 1 звездочка = " << (double)maxY/MAX_STARS << endl;  
    cout << endl;  
  
    for(int x = MIN_X; x <= MAX_X; x++) {  
        int y = x * x;  
  
        // Форматированный вывод координат  
        cout << setw(3) << x << " | y=" << setw(3) << y << "  
  
        // Вычисляем количество звездочек с масштабированием  
        int stars = (int)((double)y / maxY * MAX_STARS);  
  
        // Рисуем звездочки  
        for(int i = 0; i < stars; i++) {  
            cout << "*";  
        }  
  
        cout << endl;
```

```
    }  
  
    return 0;  
}
```

Задача 3.3: Анализ последовательности

```
#include <iostream>  
#include <climits> // для INT_MIN и INT_MAX  
using namespace std;  
  
int main() {  
    int number;  
    int count = 0;           // Количество чисел  
    int evenCount = 0;       // Количество четных чисел  
    int sum = 0;             // Сумма всех чисел  
    int maxNum = INT_MIN;    // Начальное значение - минимальное  
    int minNum = INT_MAX;    // Начальное значение - максимальное  
  
    cout << "Анализ последовательности чисел" << endl;  
    cout << "Вводите числа (0 для завершения):" << endl;  
  
    while(true) {  
        cout << "Число " << (count + 1) << ": ";  
        cin >> number;  
  
        // Проверка на окончание ввода  
        if(number == 0) {  
            break;  
        }  
  
        // Обновляем статистику  
        count++;  
        sum += number;  
  
        // Проверка на четность  
        if(number % 2 == 0) {  
            evenCount++;  
        }  
  
        // Обновляем максимум и минимум
```

```

        if(number > maxNum) {
            maxNum = number;
        }
        if(number < minNum) {
            minNum = number;
        }
    }

    // Выводим результаты
    cout << "\n=== Результаты анализа ===" << endl;

    if(count == 0) {
        cout << "Не введено ни одного числа!" << endl;
    } else {
        cout << "Количество чисел: " << count << endl;
        cout << "Сумма чисел: " << sum << endl;
        cout << "Среднее арифметическое: " << (double)sum / count << endl;
        cout << "Максимальное число: " << maxNum << endl;
        cout << "Минимальное число: " << minNum << endl;
        cout << "Количество четных чисел: " << evenCount << endl;
        cout << "Количество нечетных чисел: " << (count - evenCount) << endl;
    }

    return 0;
}

```

Уровень 4: Сложный

Задача 4.1: Игра "Угадай число"

```

#include <iostream>
#include <cstdlib> // для rand() и srand()
#include <ctime>    // для time()
using namespace std;

int main() {
    // Инициализация генератора случайных чисел
    srand(time(0));
}

```

```

// Загадываем число от 1 до 100
int secretNumber = rand() % 100 + 1;
int guess;          // Предположение игрока
int attempts = 0;   // Количество попыток
const int MAX_ATTEMPTS = 10; // Максимальное число попыток

cout << "=== Игра 'Угадай число' ===" << endl;
cout << "Я загадал число от 1 до 100." << endl;
cout << "У вас есть " << MAX_ATTEMPTS << " попыток." << endl;
cout << "===== " << endl;

// Основной игровой цикл
while(attempts < MAX_ATTEMPTS) {
    attempts++;

    cout << "\nПопытка " << attempts << "/" << MAX_ATTEMPTS << endl;
    cout << "Ваш вариант: ";
    cin >> guess;

    // Проверяем предположение
    if(guess == secretNumber) {
        cout << "\n🎉 Поздравляю! Вы угадали число " << secretNumber << endl;
        cout << " за " << attempts << " попыток!" << endl;
        break; // Выходим из цикла - игра окончена
    }
    else if(guess < secretNumber) {
        cout << "Мое число БОЛЬШЕ вашего." << endl;
    }
    else {
        cout << "Мое число МЕНЬШЕ вашего." << endl;
    }

    // Подсказка после нескольких неудачных попыток
    if(attempts == MAX_ATTEMPTS / 2) {
        if(secretNumber % 2 == 0) {
            cout << "Подсказка: число четное!" << endl;
        } else {
            cout << "Подсказка: число нечетное!" << endl;
        }
    }
}

// Если попытки закончились

```



```

if(attempts == MAX_ATTEMPTS && guess != secretNumber) {
    cout << "\n😞 Увы! Вы исчерпали все попытки." << endl;
    cout << "Загаданное число было: " << secretNumber << endl;
}

// Статистика игры
cout << "\n=== Статистика игры ===" << endl;
cout << "Загаданное число: " << secretNumber << endl;
cout << "Потрачено попыток: " << attempts << endl;

if(attempts <= 5) {
    cout << "Отличный результат!" << endl;
} else if(attempts <= 8) {
    cout << "Хороший результат!" << endl;
} else {
    cout << "Попрактикуйтесь еще!" << endl;
}

return 0;
}

```

Задача 4.2: Шахматная доска

```

#include <iostream>
using namespace std;

int main() {
    const int SIZE = 8; // Размер доски

    cout << "Шахматная доска " << SIZE << "x" << SIZE << endl;
    cout << "======" << endl;

    // Внешний цикл - строки
    for(int row = 0; row < SIZE; row++) {
        // Выводим номер строки
        cout << (SIZE - row) << " "; // В шахматах нумерация

        // Внутренний цикл - столбцы
        for(int col = 0; col < SIZE; col++) {
            // Определяем цвет клетки
            // Если сумма индексов четная - черная, нечетная

```

```

        if((row + col) % 2 == 0) {
            cout << "■ "; // Черная клетка
            // или cout << "##"; или cout << "B ";
        } else {
            cout << "□ "; // Белая клетка
            // или cout << " "; или cout << "W ";
        }
    }

    cout << endl; // Переход на новую строку
}

// Выводим буквы для столбцов
cout << " ";
for(char col = 'A'; col < 'A' + SIZE; col++) {
    cout << col << " ";
}
cout << endl;

return 0;
}

```

Расширенная версия с выбором символов:

```

#include <iostream>
using namespace std;

int main() {
    int size;
    char black, white;

    cout << "Создание шахматной доски" << endl;
    cout << "=====" << endl;

    // Запрашиваем параметры
    cout << "Введите размер доски: ";
    cin >> size;

    cout << "Введите символ для черной клетки: ";
    cin >> black;

    cout << "Введите символ для белой клетки: ";
}

```

```
cin >> white;

// Рисуем доску
cout << "\nДоска " << size << "x" << size << ":" << endl;

// Верхняя граница
cout << "┌";
for(int i = 0; i < size * 2 - 1; i++) cout << "-";
cout << "┐" << endl;

// Сама доска
for(int row = 0; row < size; row++) {
    cout << "|";

    for(int col = 0; col < size; col++) {
        if((row + col) % 2 == 0) {
            cout << black;
        } else {
            cout << white;
        }

        // Разделитель между клетками
        if(col < size - 1) cout << " ";
    }

    cout << "| " << (size - row) << endl;
}

// Нижняя граница
cout << "└";
for(int i = 0; i < size * 2 - 1; i++) cout << "-";
cout << "┘" << endl;

// Буквы столбцов
cout << " ";
for(int i = 0; i < size; i++) {
    cout << (char)('A' + i) << " ";
}
cout << endl;

return 0;
}
```

Итоговая зачетная задача: "Интеллектуальный калькулятор"

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int choice;
    bool running = true;

    cout << "=== ИНТЕЛЛЕКТУАЛЬНЫЙ КАЛЬКУЛЯТОР ===" << endl;
    cout << "===== " << endl;

    // Главный цикл программы
    do {
        // Вывод меню
        cout << "\n==== ГЛАВНОЕ МЕНЮ ===== " << endl;
        cout << "1. Сложение ряда чисел" << endl;
        cout << "2. Таблица умножения" << endl;
        cout << "3. Поиск делителей числа" << endl;
        cout << "4. Выход" << endl;
        cout << "===== " << endl;
        cout << "Выберите операцию (1-4): ";
        cin >> choice;

        // Обработка выбора
        switch(choice) {
            case 1: {
                // === Опция 1: Сложение ряда чисел ===
                cout << "\n=== СЛОЖЕНИЕ РЯДА ЧИСЕЛ ===" << endl;
                cout << "Вводите числа. Правила:" << endl;
                cout << " - 0 для завершения ввода" << endl;
                cout << " - 999 для аварийного выхода" << endl;
                cout << " - Отрицательные числа пропускаются" << endl;
                cout << "===== " << endl;

                int number;
                int sum = 0;
```

```
int count = 0;

while(true) {
    cout << "Число " << (count + 1) << ": ";
    cin >> number;

    // Проверка на аварийный выход
    if(number == 999) {
        cout << "Аварийный выход!" << endl;
        break;
    }

    // Проверка на завершение ввода
    if(number == 0) {
        cout << "Ввод завершен." << endl;
        break;
    }

    // Проверка на отрицательное число
    if(number < 0) {
        cout << "Отрицательное число пропущено" << endl;
        continue; // Переходим к следующей итерации
    }

    // Добавляем число к сумме
    sum += number;
    count++;
    cout << "Добавлено: " << number << " | Текущая сумма: " << sum << endl;
}

if(count > 0) {
    cout << "\n--- Результат ---" << endl;
    cout << "Сложено чисел: " << count << endl;
    cout << "Сумма: " << sum << endl;
    cout << "Среднее: " << fixed << setprecision(2) << (sum / count) << endl;
} else {
    cout << "Числа не введены." << endl;
}

break;
}

case 2: {
```

```

// === Опция 2: Таблица умножения ===
cout << "\n=== ТАБЛИЦА УМНОЖЕНИЯ ===" << endl;

int number;
cout << "Введите число: ";
cin >> number;

cout << "\nТаблица умножения на " << number << endl;
cout << "-----" << endl;

for(int i = 1; i <= 10; i++) {
    int result = number * i;

    cout << number << " x " << setw(2) << i << " = " << result << endl;

    // Подсвечиваем большие результаты
    if(result > 50) {
        cout << " *";
    }

    cout << endl;
}

cout << "-----" << endl;
cout << "* - результат больше 50" << endl;

break;
}

case 3: {
    // === Опция 3: Поиск делителей ===
    cout << "\n=== ПОИСК ДЕЛИТЕЛЕЙ ЧИСЛА ===" << endl;

    int number;
    cout << "Введите число: ";
    cin >> number;

    if(number == 0) {
        cout << "У нуля бесконечно много делителей" << endl;
        break;
    }

    // Берем абсолютное значение (для отрицательных чисел)

```

```

int absNumber = (number < 0) ? -number : number;

cout << "\nДелители числа " << number << ":" .
cout << "-----" << endl;

int divisorCount = 0;

// Перебираем возможные делители
for(int i = 1; i <= absNumber; i++) {
    if(absNumber % i == 0) {
        divisorCount++;

        // Проверяем, является ли делитель простым
        bool isPrime = true;
        for(int j = 2; j * j <= i; j++) {
            if(i % j == 0) {
                isPrime = false;
                break;
            }
        }

        // Выводим делитель
        cout << i;
        if(i != absNumber) cout << ", ";

        // Выделяем простые делители
        if(isPrime && i > 1) {
            cout << " (простой)";
        }

        cout << endl;
    }
}

cout << "\nВсего делителей: " << divisorCount;

// Проверяем, является ли число простым
if(divisorCount == 2) {
    cout << "Число " << number << " - ПРОСТОЕ"
}

break;
}

```

```

        case 4: {
            // === Опция 4: Выход ===
            cout << "\nСпасибо за использование калькулята\n";
            cout << "До свидания!" << endl;
            running = false;
            break;
        }

        default: {
            cout << "\nОшибка! Выберите операцию от 1 до 4\n";
            break;
        }
    }

    // Спросить о продолжении (если не выходим)
    if(running && choice != 4) {
        char continueChoice;
        cout << "\nВернуться в главное меню? (y/n): ";
        cin >> continueChoice;

        if(continueChoice != 'y' && continueChoice != 'Y')
            running = false;
        cout << "Завершение работы..." << endl;
    }
}

} while(running);

return 0;
}

```



Дополнительные функции для усложнения:

1. Проверка на палиндром:

```

bool isPalindrome(int n) {
    int original = n;

```



```
int reversed = 0;

while(n > 0) {
    int digit = n % 10;
    reversed = reversed * 10 + digit;
    n /= 10;
}

return original == reversed;
}

// Использование:
for(int i = 100; i <= 999; i++) {
    if(isPalindrome(i)) {
        cout << i << " ";
    }
}
```

2. Сумма цифр числа:

```
int sumOfDigits(int n) {
    int sum = 0;

    // Работаем с положительным числом
    n = (n < 0) ? -n : n;

    while(n > 0) {
        sum += n % 10; // Последняя цифра
        n /= 10;      // Убираем последнюю цифру
    }

    return sum;
}
```

3. Рисование разных фигур:

```
// Треугольник
for(int i = 1; i <= 5; i++) {
    for(int j = 1; j <= i; j++) {
```

```
        cout << "*";
    }
    cout << endl;
}

// Прямоугольник
int width = 10, height = 5;
for(int i = 0; i < height; i++) {
    for(int j = 0; j < width; j++) {
        cout << "*";
    }
    cout << endl;
}

// Ромб
int n = 5;
for(int i = 1; i <= n; i++) {
    for(int j = i; j < n; j++) cout << " ";
    for(int j = 1; j <= (2*i-1); j++) cout << "*";
    cout << endl;
}
for(int i = n-1; i >= 1; i--) {
    for(int j = n; j > i; j--) cout << " ";
    for(int j = 1; j <= (2*i-1); j++) cout << "*";
    cout << endl;
}
```



Советы по решению задач:

1. **Анализируйте задачу перед кодом:** что нужно получить на выходе?
2. **Разбивайте сложные задачи** на подзадачи
3. **Используйте отладку:** выводите промежуточные значения
4. **Тестируйте граничные случаи:** 0, 1, отрицательные числа
5. **Комментируйте сложные моменты** в коде

Все решения готовы к компиляции и запуску! 🚀